

AXEL v2

External Repository Integration Addendum

Companion to `axel-v2-prd.pdf`

Prepared: September 20, 2026

This document maps eight externally-sourced repositories onto Axel's existing architecture, section by section, with an honest fit assessment and explicit risk flags where a repo's design conflicts with Axel's own security principles (Chapter 11 of the main PRD) — flagged for a deliberate decision, not silently adopted.

Contents

1	Summary	2
2	Repository-by-Repository Detail	3
2.1	wensongz/stock-portfolio-manager — dashboard UI reference	3
2.2	quantopian/zipline — use the maintained fork instead	3
2.3	quant-science/sunday-quant-scientist — research input, not code	4
2.4	Anand-Swamy/sportsbetting — treat as a sketch, not a library	4
2.5	IDouble/Forex-USD-Currency-Market-Chart — Forex dashboard widget	4
2.6	dylanpersonguy/OpenCharts — reconsider the dashboard default	5
2.7	lihanyu81/polymarket_lp_tool — real strategy fit, real security conflict	5
2.8	public-clis/twitter-cli — usable read-only, flagged for write actions	6
3	Net Changes to the Main PRD	8

1. Summary

Eight repositories were reviewed against Axel’s existing architecture. None require a change to the core trust-boundary design (Agent service / Trading core / Watchdog) — each is either a UI reference, a backtesting engine substitution, a research reading list, or a section-level strategy/data source that flows through the same **Signal** → **Proposal** → Risk-package pipeline as everything already in the PRD. Two repositories raise a genuine conflict with Axel’s own security rules and are flagged rather than quietly integrated.

Repository	What it actually is	Where it fits / flag
wensongz/stock-portfolio-manager	Desktop portfolio tracker (Tauri+React+Rust+SQLite), multi-market, AI chat pane	UI/UX reference for dashboard/. No flag.
quantopian/zipline	Event-driven backtesting engine	Archived, unmaintained. Use stefan-jansen/zipline-reloaded instead.
quant-science/sunday-quant-scientist	Newsletter archive on quant/algo trading	Research input to Quant Research Agent. Not code.
Anand-Swamy/sportsbetting	Small personal sports-betting script	Rough sketch reference only; low maturity (2 stars).
IDouble/Forex-USD-Currency-Market-Chart	Static HTML/TradingView chart widgets	Drop-in Forex dashboard widget. No flag.
dylanpersonguy/OpenCharts	Full self-hosted trading terminal, backend-agnostic	Strong candidate to amend the "Grafana first" dashboard default.
lihanyu81/polymarket_lp_tool	Polymarket market-making bot (Python+Rust)	Requires a raw wallet private key — conflicts with Ch. 11 security rules.
public-clis/twitter-cli	Reads/writes X/Twitter via extracted browser cookies, not the official API	Unofficial access method; ToS/account-risk exposure.

2. Repository-by-Repository Detail

2.1 wensongz/stock-portfolio-manager — dashboard UI reference

A real, actively built desktop app for personal portfolio tracking across US/CN/HK markets, not a trading bot — it has no execution logic and nothing to reuse in **risk/** or **execution/**. Its value to Axel is purely as a **UI/UX reference** for the dashboard:

- Its Quarterly review / Performance / Review pages model the “why did it take that trade?” requirement from the main PRD’s Phase 4 exit criteria.
- Its Options page (options-specific P&L and Greeks display) is directly relevant to Axel’s Options section dashboard panel.
- Its AI Assistant chat pane is a working example of the read-only chat interface `comms/chat.py` is scoped to be in Phase 4.

Note

Integration approach: don’t import code (different language/framework). Use as a design reference when building **dashboard/** in Phase 4, particularly the Options and per-section review views.

2.2 quantopian/zipline — use the maintained fork instead

quantopian/zipline itself is **archived and explicitly unmaintained** — the original maintainers confirmed in the repo’s own issues that Quantopian folded and the team moved on. The community-maintained continuation, **stefan-jansen/zipline-reloaded**, is the version actually worth adopting.

Note

Integration approach: evaluate **zipline-reloaded** as the underlying event-driven backtesting engine for the **Stocks and Options sections specifically**. It already implements point-in-time data handling, market calendar awareness, and a pandas-based output format that fits the existing `equity_snapshots/positions_snapshots` schema. Keep Axel’s own `data/ml/validation.py` admission gate (expectancy thresholds, walk-forward enforcement, multiple-testing adjustment) as a layer *on top of* its output, rather than replacing it. Crypto and Forex still lean on the custom backtester or freqtrade’s backtesting mode, since zipline-reloaded is equities/options-oriented.

Update to Chapter 10 of the main PRD: replace `quantopian/zipline` with `stefan-jansen/zipline-reloaded` in the reference repository table.

2.3 `quant-science/sunday-quant-scientist` — research input, not code

A newsletter archive, not a runnable library. Its role is identical to `stefan-jansen/machine-learning-for-trading`'s existing role in the PRD: a source of **strategy ideas to evaluate**, never a direct dependency.

Note

Integration approach: feed candidate strategies surfaced here into the Quant Research Agent's observation loop as a manually-curated reading list, or have a human periodically mine it for playbook candidates to run through `data/ml/backtester.py`. It does not get a folder in the repo structure — it's a reading list, not a dependency.

2.4 `Anand-Swamy/sportsbetting` — treat as a sketch, not a library

A small, low-star personal project (2 stars) — a reasonable example of a first-pass sports-betting algorithm, but not production-grade code with tests, documentation, or an API contract.

Note

Integration approach: use only as a rough starting reference when writing `sections/bets_predictions/analysts/odds_movement_analyst.py` or `line_value_analyst.py`. Read its approach, don't import it as a dependency — expect to rewrite most of it to match Axel's `BaseAnalyst` contract (schema-validated `Signal` output, no side-effect tools).

2.5 `IDouble/Forex-USD-Currency-Market-Chart` — Forex dashboard widget

Static HTML/TradingView-widget snippets rendering USD-pair charts. No data pipeline, no trading logic — purely a display component.

Note

Integration approach: usable directly, or lightly adapted, as the chart widget inside the Forex section's dashboard panel once Phase 4 is underway. Low effort, low risk, cosmetic value only.

2.6 dylanpersonguy/OpenCharts — reconsider the dashboard default

The most substantial find in this list, and worth flagging against an existing decision in the main PRD.

What it is: a real, actively developed, self-hosted trading terminal — candlestick charting with a custom Canvas 2D engine, 20+ indicators, a PineScript transpiler, multi-exchange adapters (CCXT, Alpaca), WebSocket streaming, an order ticket, and an in-browser paper-trading engine. Built on Next.js + Express + PostgreSQL + Redis. Its data layer is **deliberately isolated behind two swappable modules** (a REST-shaped `api` module and a `wsClient` streaming module) specifically so it can be pointed at a real backend instead of its demo layer — explicitly documented as a supported use case (“Bring your own data”).

Clean Fit

Why this matters for Chapter 10 of the main PRD: the PRD defaults to “Grafana first, custom UI only if Grafana falls short.” OpenCharts is a serious candidate for that fallback, and may be worth pulling forward, because Grafana is metrics/logs-oriented and does not natively provide a trading-terminal-style chart/order view — which is exactly what OpenCharts already does, backend-agnostically.

Recommended amendment to Chapter 10: *Grafana for operational metrics and alerting (drawdown, exposure, agent logs) remains the fast path for Phase 4; OpenCharts is evaluated in parallel as the trading-terminal-style view (charts, order ticket, position visualization) once a section reaches Phase 5 paper soak, since that’s where a richer visual view of live paper trades starts to matter.* This is an amendment to evaluate, not a mandate to adopt.

Risk Flag

OpenCharts’ own paper-trading engine is in-browser and disconnected from Axel’s real Risk package. If adopted, it must be wired to **read** Axel’s data and **display** the Trading core’s real state, never to independently execute or simulate trades that could be confused with Axel’s actual paper-trading records. Its own demo paper engine should be disabled entirely in any Axel integration to avoid two sources of truth about “what trades happened.”

2.7 lihanyu81/polymarket_lp_tool — real strategy fit, real security conflict

What it is: an active, real Polymarket liquidity-provision / market-making tool (Python and Rust versions), with passive market monitoring, custom pricing logic, and anti-sniping parameters (mid-price jump detection, reprice throttling, fill cooldowns) — genuinely relevant prior art for a **market-making strategy candidate** in the Bets/Predictions section, which the main PRD’s Section 3.5 does not currently cover.

Risk Flag

The conflict, stated plainly: this tool requires `PRIVATE_KEY` (or `POLYMARKET_PRIVATE_KEY`) directly in its `.env` — i.e., **a raw self-custody wallet private key**, not a scoped, revocable, withdrawal-disabled API key. This is exactly the pattern Chapter 11 of the main PRD flags as materially higher risk than an exchange account with scoped API keys, and explicitly says self-custody wallets are “not recommended until the exchange-account approach has a proven operational track record.”

Recommended path, not a rejection:

1. Don’t wire this tool’s key-handling directly into Axel. Treat its market-making/pricing logic as a **reference strategy design** to reimplement inside `sections/bets_predictions/`, following Axel’s own contracts (proposes via `Proposal/AllocationProposal`, sizing through `risk/sizer.py`, execution through a proper `execution/` adapter).
2. If Polymarket market-making is pursued at all, it should go through the same **Phase 9 gate** as the rest of Bets/Predictions — legal review first — plus an explicit decision about whether a private-key-holding wallet is ever acceptable inside the Trading core’s trust boundary. This is a real open decision, flagged for Phase 9 planning, not solved here.
3. If adopted, the private key must be held **only** by the Trading core, never the Agent service, exactly like every other broker credential in Chapter 11.

2.8 `public-clis/twitter-cli` — usable read-only, flagged for write actions

What it is: a CLI that reads X/Twitter feeds, bookmarks, search, and user timelines **without the official API**, by extracting browser session cookies (`auth_token`, `ct0`) from a locally logged-in browser session. Write actions (posting, replying, quoting) additionally require full browser cookies, since token-only auth risks a platform-side “automated behavior” flag.

Two separate concerns:

1. **Read-only use** (pulling tweets/search results as a sentiment data source) is functionally similar to the other ingestion sources already scoped in the main PRD’s data chapter (NewsAPI, GDELT) — the output is just more untrusted text for the Sentiment Analyst, handled identically: never as instructions, always schema-validated before it becomes a **Signal**.
2. **The access method itself carries risk independent of Axel’s architecture:** browser-cookie extraction to bypass the official API risks the account (rate limiting, suspension) and sits in a legal/ToS gray area that Axel’s trust-boundary design cannot mitigate — this is a decision about the data source, not something the architecture controls for.

Risk Flag

Integration approach, if used at all: read-only only. Do not wire write actions into Axel under any circumstance — there is no legitimate reason for a trading system to post to social media, and doing so would put live session cookies inside a system with financial credentials nearby. If read-only ingestion is used, the extracted session cookie is itself a credential and

must be treated with the same care as a broker key: stored via the secrets manager, never in the Agent service's own file space, rotated on any suspicion. Given the ToS exposure, consider whether an official, rate-limited API (even paid) is worth the cost to avoid running a trading system's data pipeline on an unofficial access method.

3. Net Changes to the Main PRD

Chapter affected	Change
Ch. 3 (Repository Structure)	No new top-level folders required. <code>dashboard/</code> gains an evaluation note pointing to OpenCharts alongside Grafana.
Ch. 4, Bets/Predictions section	Add “Polymarket LP / market-making” as a fourth candidate strategy, explicitly gated on the Phase 9 legal review and a wallet-custody decision not yet made.
Ch. 10 (Technology Stack)	Reference repository table: replace <code>quantopian/zipline</code> with <code>stefan-jansen/zipline-reloaded</code> . Add OpenCharts as a dashboard candidate alongside Grafana.
Ch. 11 (Security)	Add an explicit note: any tool requiring a raw private key or an unofficial-access credential (browser session cookie) must be evaluated against the existing trade-only/withdrawal-disabled/scoped-API-key standard before adoption — neither <code>polymarket_lp_tool</code> ’s wallet key nor <code>twitter-cli</code> ’s session cookie currently meets that bar as-is.

Note

None of the eight repositories require a change to the core trust-boundary architecture (Chapter 2 of the main PRD). The two flagged items above are decisions for deliberate human judgment, not blockers to the rest of the integration.

This document describes software integration options only. It is not financial or legal advice.